in your application's database using ActiveRecordStore.

*session_store = :drb_store*
: DRb is a protocol that allows Ruby processes to share objects over a network connection. Using the DRbStore database manager, Rails stores session data on a DRb server (which you manage outside the web application). Multiple instances of your application, potentially running on distributed servers, can access the same DRb store. DRb uses Marshal to serialize objects.

*session_store = :mem_cache_store*
: memcached is a freely available, distributed object caching system maintained by Dormando.[105] memcached is more complex to use than the other alternatives and is probably interesting only if you're already using it for other reasons at your site.

*session_store = :memory_store*
: This option stores the session data locally in the application's memory. Because no serialization is involved, any object can be stored in an in-memory session. As we'll see in a minute, this generally isn't a good idea for Rails applications.

*session_store = :file_store*
: Session data is stored in flat files. It's pretty much useless for Rails applications because the contents must be strings. This mechanism supports the additional configuration options :prefix, :suffix, and :tmpdir.

## Comparing Session Storage Options

With all these session options to choose from, which should you use in your application? As always, the answer is "it depends."

When it comes to performance, there are few absolutes, and everyone's context is different. Your hardware, network latencies, database choices, and possibly even the weather will impact how all the components of session storage interact. Our best advice is to start with the simplest workable solution and then monitor it. If it starts to slow you down, find out why before jumping out of the frying pan.